El qubit como vector

Genesis Silesky

20 de octubre de 2026

1. El qubit como vector

1.1. Objetivo

Entender que un qubit es un vector de dos amplitudes complejas normalizado, y cómo esas amplitudes determinan las probabilidades de medición.

1.2. Conceptos previos

Concepto previo: lo que hace falta para este capítulo

Ver el capítulo de prerrequisitos (sección **??**): números complejos (módulo y fase), vectores y norma, notación de Dirac y regla de Born. Si solo se recuerda una cosa: el *módulo al cuadrado* de una amplitud es una probabilidad.

1.3. Fundamento teórico

Un bit clásico vale 0 o 1. Un qubit es un vector

$$|\psi\rangle = \alpha\,|0\rangle + \beta\,|1\rangle = \begin{pmatrix}\alpha\\ \beta\end{pmatrix}, \qquad \alpha,\beta\in\mathbb{C}, \quad |\alpha|^2+|\beta|^2 = 1. \tag{1}$$

Por la regla de Born, al medir se obtiene 0 con probabilidad $|\alpha|^2$ y 1 con probabilidad $|\beta|^2$. La condición de normalización asegura que las probabilidades sumen 1. Dos observaciones:

- **Fase global:** $e^{i\gamma}\,|\psi\rangle$ y $|\psi\rangle$ son físicamente indistinguibles.
- **Fase relativa:** la fase de β respecto a α no altera las probabilidades de esta medición, pero sí cómo interfiere el qubit con otros después (capítulo 05a).

Salvo fase global, todo qubit se parametriza como $\cos\frac{\theta}{2}\,|0\rangle + e^{i\varphi}\sin\frac{\theta}{2}\,|1\rangle$ (esfera de Bloch).

Ejemplo del código. Se toma $\alpha = 1/\sqrt{3}$ y $\beta = \sqrt{2/3}\,e^{i\pi/4} = (1+i)/\sqrt{3}$. Entonces $|\alpha|^2 = 1/3$ y $|\beta|^2 = 2/3$, por lo que $P(0) = 1/3$, $P(1) = 2/3$.

1.4. Código (completamente comentado)

```
# %% [markdown]
# # 01 - El qubit como vector
# |psi> = alpha|0> + beta|1>, con |alpha|^2 + |beta|^2 = 1

# %% Celda 1: importaciones
# numpy nos da numeros complejos y funciones matematicas (sqrt, exp, pi).
import numpy as np
```

```python

# Statevector representa el estado de un sistema cuantico como un vector
# de amplitudes complejas (el "vector de estado").
from qiskit.quantum_info import Statevector

# %% Celda 2: los estados de la base computacional
# from_label("0") construye |0>, que en forma de vector columna es (1, 0).
# Estos dos vectores son la base canonica: cualquier qubit es una
# combinacion lineal de ellos.
print("|0> =", Statevector.from_label("0").data)  # [1+0j, 0+0j]
print("|1> =", Statevector.from_label("1").data)  # [0+0j, 1+0j]

# %% Celda 3: un qubit arbitrario
# Elegimos dos amplitudes complejas alpha y beta.
# alpha es real; beta tiene modulo sqrt(2/3) y una fase de pi/4 radianes.
# La fase e^{i*pi/4} NO cambia la probabilidad de medir, pero si cambia
# como interfiere el qubit con otros (ver el ejemplo 05a).
alpha = 1 / np.sqrt(3)
beta = np.sqrt(2 / 3) * np.exp(1j * np.pi / 4)

# Construimos el estado a partir de la lista de amplitudes [alpha, beta].
psi = Statevector([alpha, beta])

# .data devuelve el arreglo de numpy con las amplitudes.
print("Vector:", psi.data)

# Condicion de normalizacion: la suma de los modulos al cuadrado debe ser 1.
# np.abs de un complejo es su modulo; el 1.0000000000000002 que se imprime
# es error de redondeo de punto flotante, no un error del estado.
print("Norma |a|^2+|b|^2 =", np.sum(np.abs(psi.data) ** 2))

# %% Celda 4: de amplitudes a probabilidades (regla de Born)
# probabilities_dict() aplica la regla de Born: P(k) = |amplitud_k|^2.
# Esperamos P(0) = 1/3 y P(1) = 2/3. Convertimos a float para imprimir limpio.
probs = {str(k): round(float(v), 4) for k, v in psi.probabilities_dict().items()}
print("Probabilidades:", probs)

# %% Celda 5: un vector NO normalizado no es un estado valido
# (1, 1) tiene norma^2 = 2, asi que no representa un qubit fisico.
# is_valid() comprueba justamente que la norma sea 1.
print("(1,1) es valido?:", Statevector([1, 1]).is_valid())  # False
print("(1,1)/sqrt(2) es valido?:", Statevector([1, 1] / np.sqrt(2)).is_valid())  # True
```

1.5. Explicación celda por celda

Celda 1 Importa `numpy` (complejos y funciones) y `Statevector`, la clase que guarda las amplitudes.

Celda 2 Construye $|0\rangle = (1, 0)$ y $|1\rangle = (0, 1)$ con `from_label`. Son la base canónica.

Celda 3 Define α, β y crea el estado. La suma $|\alpha|^2 + |\beta|^2$ sale 1.0000000000000002 y no exactamente 1: es error de redondeo de punto flotante (aritmética de doble precisión).

Celda 4 `probabilities_dict()` aplica la regla de Born. Se convierte a `float` y `str` para que la impresión sea legible (NumPy 2 imprime sus tipos propios).

Celda 5 Muestra que $(1, 1)$ no es válido (norma$^2 = 2$) y que $(1, 1)/\sqrt{2}$ sí lo es.

1.6. Salida obtenida

```
|0> = [1.+0.j 0.+0.j]
|1> = [0.+0.j 1.+0.j]
Vector: [0.57735027+0.j         0.57735027+0.57735027j]
Norma |a|^2+|b|^2 = 1.0000000000000002
Probabilidades: {'0': 0.3333, '1': 0.6667}
(1,1) es valido?: False
(1,1)/sqrt(2) es valido?: True
```

Esta salida es determinista (no hay medición).

1.7. Interpretación

Las amplitudes son números complejos, pero las probabilidades salen reales y suman 1. La fase de β ($\pi/4$) no aparece en las probabilidades: es información que solo se revela mediante interferencia.

1.8. Ejercicios

1. Cambie la fase de β a π y verifique que las probabilidades no cambian.
2. Construya un estado con $P(1) = 0.9$ y compruébelo.
3. ¿Es $(\frac{3}{5}, \frac{4}{5}i)$ un estado válido? Verifíquelo con `is_valid()`.

1.9. Errores comunes

- Olvidar normalizar: Qiskit no lo hace automáticamente y `Statevector` podría ser inválido.
- Confundir amplitud con probabilidad: la amplitud puede ser negativa o compleja; la probabilidad es su módulo al cuadrado.